

Iterated Closest Point (ICP) for Scan Matching in SLAM Applications

Oliver MacDonald

April 30, 2025

Abstract

This paper investigates the mathematical foundations, implementation, and evaluation of Iterative Closest Point (ICP) algorithms for scan matching, a core component of SLAM (Simultaneous Localization and Mapping) in autonomous robotic systems. Variants including point-to-point, point-to-plane, and Generalized ICP are examined with respect to accuracy, convergence, and computational cost in the context of edge device deployment for lunar robotics.

1 Introduction

Autonomous navigation is a cornerstone of modern robotics, especially in environments where external positioning systems like GPS are unavailable. This challenge is exemplified by the NASA Lunabotics competition, in which Boise State University’s Bender Robotics Team is competing with their fully autonomous lunar excavation robot, *Bender*. The robot is tasked with navigating rough, dusty terrain, collecting regolith, and returning it to a specified location—entirely without human intervention.

Bender employs a multi-sensor architecture that includes 3D LiDAR, stereo cameras, an IMU, and a Vive beacon system for indoor positional tracking. Despite this advanced suite, the navigation stack has exhibited failure modes, particularly in odometry and mapping. These failures manifest as “tornado mode,” in which accumulated error causes the map to rotate around the robot, resulting in incorrect obstacle detection and incorrect positioning. This ultimately forces the robot to make incorrect navigation decisions, such as turning in place indefinitely or abandoning its planned path.

In GPS-denied environments like the lunar surface, correcting this drift without an external reference requires accurate scan matching between consecutive LiDAR frames. Iterative Closest Point (ICP) algorithms offer a compelling solution, using geometric relationships between overlapping point clouds to estimate the robot’s motion and update the map. However, ICP comes with trade-offs between computational cost and alignment accuracy, which are especially relevant for embedded platforms like the ESP32, Teensy 4.1, and Raspberry Pi 5, where real-time constraints demand sub-10ms compute cycles.

The objective of this work is to analyze and compare three major variants of the ICP algorithm—Point-to-Point, Point-to-Plane, and Generalized ICP—focusing on their mathematical formulation, numerical stability, and suitability for real-time implementation on constrained hardware. The aim is to determine which variant provides the best balance of performance and computational cost for potential deployment on the Bender robot during the final testing phase.

The project contribution is twofold:

- A structured evaluation of ICP variants using both Python prototypes and C-based embedded implementations.

- A set of recommendations for deploying accurate, lightweight scan matching algorithms for real-time SLAM on edge robotics platforms.

2 Formulation

2.1 Rigid Transformations in 3D

A rigid body transformation T in 3D applies a rotation $R \in SO(3)$ and translation $\mathbf{t} \in \mathbb{R}^3$ to a point $\mathbf{p} \in \mathbb{R}^3$:

$$T(\mathbf{p}) = R\mathbf{p} + \mathbf{t}$$

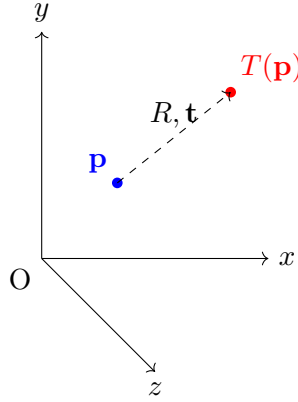


Figure 1: Rigid transformation in 3D showing rotation R and translation $\mathbf{t}$.

Rigid transformations preserve distances and angles, which is crucial for physical scan alignment tasks.

2.2 Point Cloud Registration

Given a source point cloud $\mathcal{P} = \{\mathbf{p}_i\}$ and a target (map) point cloud $\mathcal{Q} = \{\mathbf{q}_i\}$, ICP attempts to find the transformation $T = (R, \mathbf{t})$ that minimizes alignment error:

$$T^* = \arg \min_{R, \mathbf{t}} \sum_i \|\mathbf{q}_i - (R\mathbf{p}_i + \mathbf{t})\|^2$$

This assumes point correspondences $(\mathbf{p}_i, \mathbf{q}_i)$ are known or estimated via nearest neighbor search. Variants modify the error metric, correspondence criteria, and optimization strategy.

2.3 Least Squares & SVD for Best-Fit Transform

For point-to-point ICP, once correspondences are determined, we solve for T^* by minimizing the Frobenius norm of residuals. Centering both point sets (by subtracting their means) leads to an orthogonal Procrustes problem:

$$\min_R \|R\mathbf{P}_c - \mathbf{Q}_c\|_F^2$$

This is solved using Singular Value Decomposition (SVD) on the cross-covariance matrix $H = \sum_i \mathbf{p}_i \mathbf{q}_i^\top$. Let $H = U\Sigma V^\top$, then $R = VU^\top$ ensures the closest orthogonal matrix in the least squares sense.

2.4 Overdetermined Systems & QR Decomposition

For point-to-plane ICP, we minimize the orthogonal distance from a transformed point to the tangent plane of its correspondence:

$$E = \sum_i \left[\mathbf{n}_i^\top (\mathbf{q}_i - R\mathbf{p}_i - \mathbf{t}) \right]^2$$

This is linearized using small-angle approximations and solved via the least squares method:

$$Ax = b \quad \text{where} \quad A \in \mathbb{R}^{N \times 6}, \quad x = [\delta\theta_x, \delta\theta_y, \delta\theta_z, t_x, t_y, t_z]^\top$$

The overdetermined system is efficiently solved with QR decomposition or normal equations. Your implementation supports both QR and SVD-based updates depending on the ICP variant.

2.5 Covariance & Probabilistic Models in G-ICP

In Generalized ICP, each point in the source and target cloud is treated as a Gaussian-distributed random variable:

$$\mathbf{p}_i \sim \mathcal{N}(\hat{\mathbf{p}}_i, C_p), \quad \mathbf{q}_i \sim \mathcal{N}(\hat{\mathbf{q}}_i, C_q)$$

The goal is to minimize the Mahalanobis distance between transformed source points and their targets:

$$T^* = \arg \min_T \sum_i (\mathbf{q}_i - T\mathbf{p}_i)^\top (C_q + RC_p R^\top)^{-1} (\mathbf{q}_i - T\mathbf{p}_i)$$

The covariance matrices C_p and C_q describe local surface uncertainty, estimated via PCA in a local neighborhood.

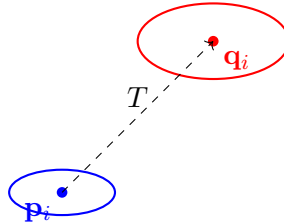


Figure 2: Covariance ellipses for a source-target pair in G-ICP. Uncertainty is modeled via local PCA.

Unlike standard ICP, G-ICP weighs the alignment based on both structural orientation and measurement noise, resulting in a more robust and symmetric scan matching framework [Segal et al., 2009].

3 ICP Algorithm Variants

3.1 Point-to-Point ICP

Point-to-point ICP seeks the transformation $T = (R, \mathbf{t})$ that minimizes the sum of squared Euclidean distances between corresponding points in the source and target point clouds:

$$E(R, \mathbf{t}) = \sum_{i=1}^N \|\mathbf{q}_i - (R\mathbf{p}_i + \mathbf{t})\|^2$$

This approach assumes that all point correspondences are valid and carries the risk of local minima when the overlap is poor. The optimal transformation is solved via SVD on the cross-covariance matrix between centered point clouds [Donoso et al., 2017].

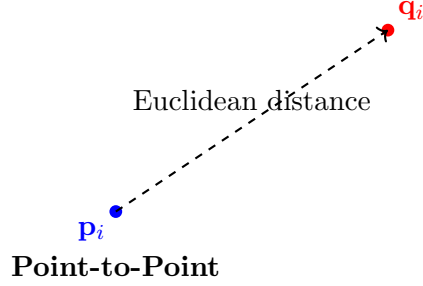


Figure 3: Point-to-Point ICP minimizes direct Euclidean distance between corresponding points.

3.2 Point-to-Plane ICP

Point-to-plane ICP leverages local surface normals to minimize the distance from each source point to the tangent plane at the corresponding target point. The objective function becomes:

$$E(R, \mathbf{t}) = \sum_{i=1}^N \left[\mathbf{n}_i^\top (\mathbf{q}_i - (R\mathbf{p}_i + \mathbf{t})) \right]^2$$

This leads to a linear least squares system solved via QR decomposition. It tends to converge faster than point-to-point ICP, especially when surface geometry is well-defined [Guevara et al., 2024].

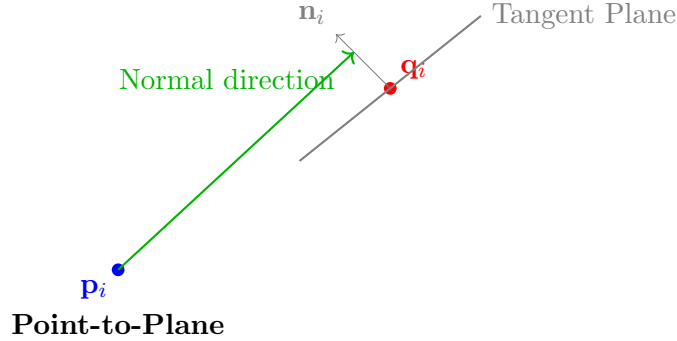


Figure 4: Point-to-Plane ICP minimizes distance along the normal vector of the target surface.

3.3 Generalized ICP

Generalized ICP (G-ICP) fuses point-to-point and point-to-plane ideas into a probabilistic framework. Each point is modeled as a Gaussian with associated covariance:

$$E(T) = \sum_{i=1}^N (\mathbf{q}_i - T\mathbf{p}_i)^\top (C_q + RC_p R^\top)^{-1} (\mathbf{q}_i - T\mathbf{p}_i)$$

This formulation improves robustness to noise and outliers by weighting residuals based on local uncertainty. It reduces sensitivity to poor correspondences and unifies multiple ICP variants under a common model [Segal et al., 2009].

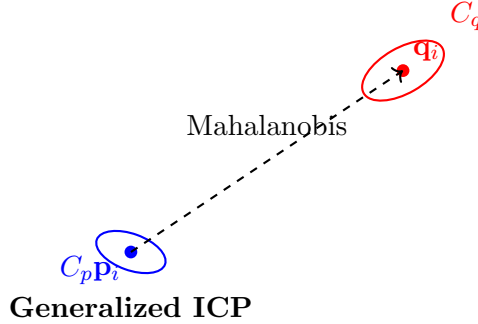


Figure 5: G-ICP minimizes Mahalanobis distance using local point covariances.

3.4 Comparison Framework

To assess the practical trade-offs of ICP variants, we compare them using:

- **Metrics:** Accuracy (final alignment error), Precision (variance over trials), and Runtime (ms/frame)
- **Datasets:** Both synthetic point clouds (e.g., shapes, noisy transforms) and real-world terrain data
- **Target Platforms:** Raspberry Pi 5, Teensy 4.1, and ESP32 for embedded deployment constraints

4 Implementation

4.1 Python Implementation Overview

4.2 C/PlatformIO Version Overview

4.3 Edge Deployment Strategy

4.4 Test Scenarios and Parameters

5 Results

5.1 Accuracy and Drift Over Time

5.2 Runtime Performance

5.3 Convergence Behavior

5.4 Hardware-Specific Observations

6 Conclusion

- Summary of findings

- Recommended ICP variant for lunar robotics
- Future work: robust data association, SLAM loop closure

References

- F. A. Donoso, K. J. Austin, and P. R. McAree. How do icp variants perform when used for scan matching terrain point clouds? *Robotics and Autonomous Systems*, 87:147–161, 2017. doi: 10.1016/j.robot.2016.10.011.
- J. Guevara, J. Gené-Mola, E. Gregorio, and F. A. Auat Cheein. A systematic analysis of scan matching techniques for localization in dense orchards. *Smart Agricultural Technology*, 9:100607, 2024. doi: 10.1016/j.atech.2024.100607.
- Aleksandr Segal, Dirk Haehnel, and Sebastian Thrun. Generalized-icp. In *Robotics: Science and Systems (RSS)*, pages 161–168. MIT Press, 2009. URL <https://doi.org/10.7551/mitpress/8727.003.0022>.

A Programs Used

- Python: NumPy, SciPy, Open3D
- C: Eigen, PlatformIO libraries for Teensy and Raspberry Pi
- Visualization: Matplotlib for Python, custom C++ visualization using OpenGL